turbo pulse documentation

Clemens Abraham (clemens.abraham@uni-rostock.de)

2026-05-29

Welcome to the world of `turbo`!

This document explains how to use the `turbo` functions to fit bioturbation models to tracer data obtained from a luminophore tracer pulse experiment. Though cited in literature the `turbo` package is not available in R easily. It can be found on github (https://github.com/r-forge/bioturbation) and made available by using git bash. However the functions used in this documentation are taken from the original github code and adopted to the special case of a luminophore pulse experiment. This markdown-document comes with a data set "luminophore.txt" and a file "turbo pulse functions.RData". Make sure all files live in the same folder.

Load packages and functions

First, install the packages if you are missing some. The required packages are the following.

```
# install packages via install.packages() if necessary
# load packages
library(deSolve)
library(rootSolve)
library(coda)
library(FME)
library(tidyverse)
library(Matrix)
library(plotrix)
```

The customized functions are loaded into the global environment from the RData-file.

```
# load customized functions
load(file = "turbo pulse functions.RData")
```

Data import and parameter setting

Data must be a table of the following format. `start` and `end` are the slice limits in cm. `lum` is the number of luminophore particles found in each layer. In practice the unit of `lum` depends on the experimental method. If you are using a volume dependent unite like RFU the concentration conversion must be skipped.

```
##   profile start end lum
## 1       a   0.0 0.5 498
## 2       a   0.5 1.0 140
## 3       a   1.0 1.5 137
## 4       a   1.5 2.0  51
## 5       a   2.0 2.5  61
## 6       a   2.5 3.0  70
```

Load your data and specify the profile you want to model. Profile "a" from the luminophore data set is used as an example here. You can also loop over all unique profile names to model all profiles automatically. The parameters are set manually. `db`, `sl` and `wt` are parameter guesses that are used as initial values by the

model fitting algorithms. `times` is the time sequence for which model output is wanted. It contains the initial time (0) and the end time (`days`).

```
# data
data <- read.table("luminophore.txt", header = TRUE)
data <- subset(data, profile == "a") # select the profile here

# global parameters
parms <- list(
  depth = max(data$end), # depth of the sediment core
  days = 14,             # observation time (pulse - slicing) in days
  dx = 0.1,              # thickness of model layers
  cakethickness = 0.2,   # thickness of the initial tracer layer
  db = 0.01,             # biodiffusion coefficient
  sl = 2,                # typical step length
                         # (standard deviation of step length distribution)
  wt = 20                # waiting time in days (expected value)
)

parms$slicenumber <- parms$depth/parms$dx # number of modeled slices
parms$times <- c(0, parms$days)           # times vector
```

Basic functions

The following functions calculate basic parameters of both the measured profile and the initial profile.

```
# observed profile
datalimits       <- unique(c(data$start, data$end))
datamidpoints    <- midpoints(datalimits)
conc_profile     <- numtoconc(data$lum, datalimits)
dataprofileframe <- data.frame(depth = datamidpoints,
                               concentration = conc_profile)
```

`dataprofileframe` format:

```
##   depth concentration
## 1  0.25           996
## 2  0.75           280
## 3  1.25           274
## 4  1.75           102
## 5  2.25           122
## 6  2.75           140
```

```
# initial profile
initprof         <- initialprofile(conc_profile, datalimits, parms$slicenumber,
                                   parms$dx, parms$cakethickness)
initlimits       <- seq(0, parms$slicenumber*parms$dx, by = parms$dx)
initmidpoints    <- midpoints(initlimits)
initprofileframe <- data.frame(depth = initmidpoints, concentration = initprof)
```

`initprofileframe` format:

```
##   depth concentration
## 1  0.05          5010
## 2  0.15          5010
## 3  0.25             0
## 4  0.35             0
```

```
## 5  0.45             0
## 6  0.55             0
```

Diffusive model

The diffusive model fits `diffusiveobjective` to the observed profile by adjusting `db` so that the weighed sum of squared residuals is minimized. The model calculates the diffusive flux across each layer starting with the initial profile at t = 0. `ode.band()` is used to solve the resulting ordinary differential equations for every time given in `times`. `db` is returned for every fit.

```
# fit diffusive model
# lower = lower bounds on db
fit <- modFit(p=parms$db,f=diffobjective,lower=c(0))
```

```
## db: 0.01
## db: 0.01
## db: 0.01
## db: 0.0144998
## db: 0.01449981
## db: 0.01763604
## db: 0.01763604
## db: 0.0195685
## db: 0.0195685
## db: 0.0206743
## db: 0.0206743
## db: 0.02127231
## db: 0.02127231
## db: 0.02160181
## db: 0.02160181
## db: 0.02178185
## db: 0.02178185
## db: 0.02186943
## db: 0.02186943
## db: 0.02191613
## db: 0.02191613
## db: 0.02193844
## db: 0.02193844
## db: 0.02196001
## db: 0.02196001
## db: 0.02195953
## db: 0.02195953
## db: 0.02195996
## db: 0.02196001
## db: 0.02196002
## db: 0.02196
```

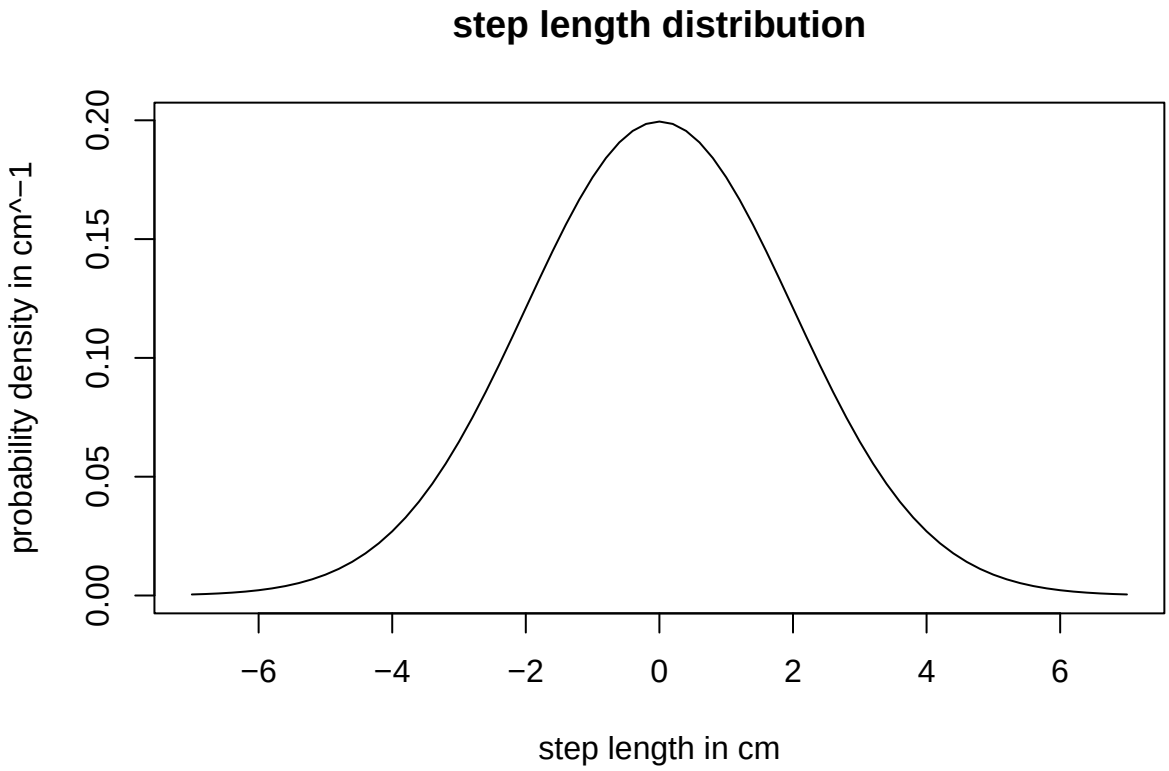
```
fitdb <- as.numeric(fit$par[1])

diff_modelprofileframe <- modelprofileframe_diff(fitdb)
```

Non local model

The non local model fits `nlobjective_analytic` the same way by adjusting the non local parameters `sl` and `wt`. Particle displacement is simulated by a continuous time random walk model (CTRW model). See Meysman et al. (2008) for further explanation. `sl` is the typical step length of a particle that is being displaced due to a bioturbation event. It characterizes the standard deviation of the normal distributed step

length. The assumption of the step length as a normal distributed variable is a fundamental assumption that underlies this implementation of the CTRW model.

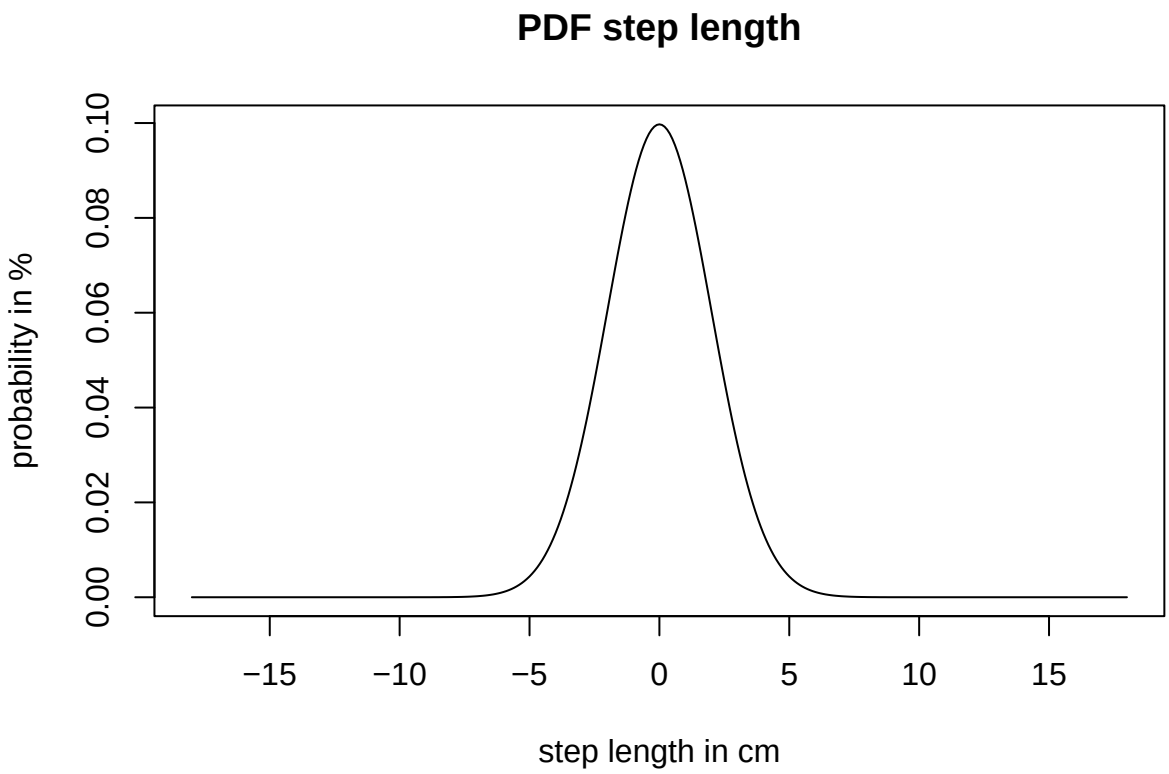


What is needed to calculate the transition rates between the model layers is the probability density function of the step length. Therefore the relevant jumping intervals are calculated and in the next step the probabilities are calculated by integration of the distribution function. Integration limits are the lower boundary `i` taken from `jump_intervals` and the upper boundary `i + dx`. Division by `wt` returns the transition rate. `values` is a numeric vector containing transition rates for every possible particle displacement. The longer the jump distance the lower the chance that it takes place. Note the similarity between the distribution function and the probability density function (PDF).

```
jump_intervals <- seq(-(parms$slicenumber + 0.5) * parms$dx,
                      (parms$slicenumber - 0.5) * parms$dx,
                      by = parms$dx)

values <- NULL
for (i in jump_intervals) {
  values <- c(values, integrate(psil_test, i, i + parms$dx)$value / parms$wt)
}

plot(jump_intervals+0.5*parms$dx, values*100, main = "PDF step length",
     xlab = "step length in cm", ylab = "probability in %", type = "l")
```



`nlobjective_analytic` constructs a transition matrix (`slicenumber` x `slicenumber`) that is filled with `values` column wise so that `values` is centered around the main diagonal. See Shull (2001) for the use of transition matrices in bioturbation models. `sl` and `wt` are returned for every fit.

```
# fit non local model
# lower = lower bounds on sl and wt
fitnl <- modFit(p = c(sl = parms$sl, wt = parms$wt), f = nlobjective_analytic,
                lower = c(0,0))
```

```
## step length: 2 | waiting time: 20
## step length: 2 | waiting time: 20
## step length: 2 | waiting time: 20
## step length: 2 | waiting time: 20
## step length: 1.316823 | waiting time: 13.08404
## step length: 1.316823 | waiting time: 13.08404
## step length: 1.316823 | waiting time: 13.08405
## step length: 1.412906 | waiting time: 13.03006
## step length: 1.412906 | waiting time: 13.03006
## step length: 1.412906 | waiting time: 13.03006
## step length: 1.41711 | waiting time: 13.02871
## step length: 1.41711 | waiting time: 13.02871
## step length: 1.41711 | waiting time: 13.02871
## step length: 1.417237 | waiting time: 13.02923
## step length: 1.417237 | waiting time: 13.02923
## step length: 1.417237 | waiting time: 13.02923
## step length: 1.417241 | waiting time: 13.02925
## step length: 1.417241 | waiting time: 13.02925
```

```
## step length: 1.417241 | waiting time: 13.02925
## step length: 1.417241 | waiting time: 13.02925
## step length: 1.417241 | waiting time: 13.02925
## step length: 1.417241 | waiting time: 13.02924
```

```r
fitsl <- as.numeric(fitnl$par[1]) # fitted typical step length
fitwt <- as.numeric(fitnl$par[2]) # fitted waiting time
fitnldb <- fitsl^2/(2*fitwt)      # calculates db (sl^2/2*wt)

nl_modelprofileframe <- modelprofileframe_nl(sl = fitsl, wt = fitwt)
```

Results

The results can be shown in one single plot. `Db` is given in cm^2/day.